جزوه درجه یک سیستم عامل

فصل ۳ - بخش ۷: مثال‌هایی از سیستم‌های IPC (بخش ب)

در جزوه قبلی (بخش ۷-الف)، ما پیاده‌سازی حافظه مشترک پوزیکس و عبور پیغام در Mach را دیدیم و به معرفی ALPC ویندوز رسیدیم. حالا بحث را از همانجا ادامه می‌دهیم.

ادامه ۳-۷-۳: ویندوز و ALPC (جزئیات کلیدی)

ما یاد گرفتیم که ویندوز از مدل "عبور پیغام" به نام ALPC استفاده می‌کند که در آن یک "کارفرما" از طریق "پورت‌های اتصال" به "کارگزار" وصل می‌شود و یک "پورت ارتباط" خصوصی دریافت می‌کند.

اما سوال مهم این است: وقتی داده‌ها زیاد باشند، آیا ویندوز آن‌ها را "کپی" می‌کند؟ (کاری که گفتیم کند است).

پاسخ (و نکته امتحانی): ویندوز بسیار هوشمندانه عمل می‌کند و **هر دو مدل IPC** را با هم ترکیب می‌کند:

1. **برای پیغام‌های کوچک (کمتر از ۲۵۶ بایت):** بله، پیغام مستقیماً در صف پورت **کپی** می‌شود. سریع و ساده است.
2. **برای پیغام‌های بزرگتر:** ویندوز از یک "شئ بخش" **(Section Object)** استفاده می‌کند. این شئ در واقع **"حافظه مشترک" (Shared Memory)** است!
    - یعنی به جای کپی کردن داده، کارگزار یک "تخته سیاه" (حافظه مشترک) ایجاد می‌کند و فقط "آدرس" آن را در یک پیغام کوچک برای کارفرما می‌فرستد.
1. **برای داده‌های خیلی بزرگ:** ویندوز یک API مخصوص دارد که به کارگزار اجازه می‌دهد مستقیماً در فضای آدرس کارفرما داده بنویسد/بخواند.

نتیجه‌گیری: ALPC ویندوز یک سیستم "هیبریدی" است. از "عبور پیغام" برای هماهنگی و ارسال دستورات (پیغام‌های کوچک) و از "حافظه مشترک" برای انتقال داده‌های حجیم (کارایی بالا) استفاده می‌کند.

۳-۷-۴: لوله‌ها (Pipes)

"لوله" (Pipe) یکی از قدیمی‌ترین، ساده‌ترین و در عین حال پرکاربردترین مکانیزم‌های IPC در سیستم‌عامل‌های یونیکس است.

تشبیه: لوله دقیقاً مثل یک "لوله آب" (مَجرا) عمل می‌کند.

- یک پردازه (تولیدکننده) داده‌ها را از یک سر لوله "می‌ریزد" (Write).
- پردازه دیگر (مصرف‌کننده) داده‌ها را از انتهای دیگر لوله "می‌خواند" (Read).

ما دو نوع لوله داریم:

1. لوله معمولی (Ordinary Pipe) یا بی‌نام (Anonymous Pipe)
2. لوله با نام (Named Pipe)

۳-۷-۴-۱: لوله‌های معمولی *(Ordinary Pipes)*

این لوله‌ها، نسخه ساده و کلاسیک هستند.

ویژگی‌های کلیدی:

- **یک‌طرفه (One-way):** داده‌ها فقط در یک جهت حرکت می‌کنند (از نویسنده به خواننده). اگر ارتباط دوطرفه بخواهید، باید دو لوله مجزا بسازید.
- **نیاز به رابطه والد-فرزندی:** این **مهم‌ترین محدودیت** است. لوله معمولی ***فقط*** می‌تواند بین یک پردازه والد و پردازه فرزندش (که بعد از ساخت لوله `fork` شده) استفاده شود. پردازه‌های غریبه نمی‌توانند از آن استفاده کنند.
- **عمر موقت:** لوله تا زمانی وجود دارد که پردازه‌ها از آن استفاده می‌کنند. وقتی هر دو پردازه خاتمه یابند، لوله هم از بین می‌رود.

چطور در یونیکس کار می‌کند؟ (بسیار مهم)

1. **ساخت لوله:** والد تابع `pipe(int fd[2])` را صدا می‌زند.
   - سیستم عامل یک لوله ایجاد می‌کند و دو "توصیف‌گر فایل" (File Descriptor) به والد می‌دهد:
     - `fd[0]`: انتهای خواندنی لوله (Read End)
     - `fd[1]`: انتهای نوشتنی لوله (Write End)
1. **ایجاد فرزند:** والد `fork()` را صدا می‌زند.
   - فرزند **کپی** توصیف‌گرهای فایل (`fd[0]` و `fd[1]`) را به ارث می‌برد. حالا هم والد و هم فرزند به هر دو انتهای لوله دسترسی دارند.
1. **"ترفند بستن" (The Close Trick):**
   - حالا باید مشخص کنیم چه کسی می‌نویسد و چه کسی می‌خواند. فرض کنیم والد می‌خواهد بنویسد و فرزند بخواند:
   - **والد (نویسنده):** به انتهای خواندنی (`fd[0]`) نیازی ندارد، پس آن را `close(fd[0])` می‌کند. سپس در `fd[1]` می‌نویسد.
   - **فرزند (خواننده):** به انتهای نوشتنی (`fd[1]`) نیازی ندارد، پس آن را `close(fd[1])` می‌کند. سپس از `fd[0]` می‌خواند.
1. **پایان کار:** وقتی والد نوشتن را تمام کرد، باید `close(fd[1])` کند. این کار یک سیگنال EOF (End of File) به فرزند می‌فرستد تا فرزند بفهمد که داده‌ها تمام شده‌اند.

در ویندوز: به لوله‌های معمولی، "لوله بی‌نام" (Anonymous Pipe) گفته می‌شود و دقیقاً همین رفتار (یک‌طرفه، والد-فرزندی) را دارد، هرچند پیاده‌سازی آن با `CreatePipe()` پیچیده‌تر است.

۳-۷-۴-۲ : لوله‌های با نام *(Named Pipes)*

این لوله‌ها برای رفع محدودیت‌های لوله‌های معمولی ساخته شدند.

تشبیه: لوله معمولی مثل "شیلنگ آب" خصوصی بین شما و فرزندتان است. لوله با نام مثل یک "شیر آب عمومی" در پارک است که هر کسی "آدرس" (اسم) آن را بلد باشد، می‌تواند از آن استفاده کند.

ویژگی‌های کلیدی (تفاوت‌ها):

- **نام دارد:** یک "لوله با نام" به عنوان یک فایل در سیستم فایل (مثلاً در `/tmp/my_pipe`) ساخته می‌شود.
- **بدون نیاز به والد-فرزندی:** هر پردازه "غریبه‌ای" که "نام" لوله را بداند و اجازه دسترسی داشته باشد، می‌تواند آن را باز کند و در آن بنویسد یا بخواند.

- **عمر طولانی:** لوله با نام می‌تواند حتی پس از پایان کار پردازه‌ها در سیستم باقی بماند (تا زمانی که صریحاً حذف نشود).
- **قدرتمندتر:** معمولاً "دوطرفه" (Bi-directional) هستند.
- **(در ویندوز):** لوله‌های با نام در ویندوز بسیار قدرتمندترند و حتی می‌توانند بین دو کامپیوتر مختلف در **شبکه** ارتباط برقرار کنند و هم به صورت "بایتی" (مثل شیلنگ) و هم "پیغامی" (مثل نامه) داده منتقل کنند.

لوله‌ها در عمل: جادوی خط فرمان |

تا به حال در خط فرمان لینوکس دستوری مثل `ls | more` را تایپ کرده‌اید؟

- `ls`: لیستی از فایل‌ها را نشان می‌دهد.
- `more`: متن را صفحه‌بندی می‌کند.
- **| (کاراکتر Pipe):** این کاراکتر به "پوسته" (Shell) دستور می‌دهد که:
  1. یک **لوله معمولی** ایجاد کند.
  2. خروجی استاندارد (Standard Output) دستور `ls` را به **انتهای نوشتنی** لوله (`[1]fd`) وصل کند.
  3. ورودی استاندارد (Standard Input) دستور `more` را به **انتهای خواندنی** لوله (`[0]fd`) وصل کند.

نتیجه: خروجی `ls` به جای نمایش روی صفحه، مستقیماً به ورودی `more` "لوله" می‌شود. این یک مثال کلاسیک و روزمره از قدرت IPC است!